



CellStrat

Module 1:

Claude Code Certified Architect

CCA · A CellStrat Full-Day Intensive

Arjun and His Transformation



Sourav
AI Developer Intern

His pain points



He can call an LLM from a notebook, but has never taken one to a typed, tested, secure service.

He uses autocomplete daily, but does not trust multi-file agent edits he cannot verify.

His transformation



By the end of this intensive, **Sourav** directs Claude Code like an architect: spec first, agent-built, test-verified, secured and shipped.

Spec in. Tested service out.

Your Day at a Glance

One day. Five modules. One real PoC: DischargeFlow.



Module 1

- Orient & Architect
- CLAUDE.md
- Plan Mode & Spec

MODULE OUTCOME

A CLAUDE.md constitution and a reviewed architecture built from SPEC.md



Modules 2-3

- Build & AI Features
- Claude + RAG
- Security & PHI

MODULE OUTCOME

A grounded, hardened API with a clinician sign-off workflow



Module 4

- Debug & Assure
- Checkpoints
- Tests & Hooks

MODULE OUTCOME

A debugged PoC with a test per requirement and hooks that enforce green



Module 5

- Agent & MCP
- Streamlit UI
- Docs & Ship

MODULE OUTCOME

An agent with memory, a clinician console, handoff docs and a gap audit — then the graded exam

Module 1:

Getting Started with Claude Code & AI Tools

Module Agenda + Timebox

PART 1

~ 30 min

- The AI Coding Landscape
- AI Tools
- How To Choose Right Tool

PART 2 min

~ 40

- Getting Started with Claude Code
Demo + Hands-on
- Interface orientation
- CLAUDE.md
- Key commands

PART 3

~ 50 min

- Prompt Engineering for Code Generation
Hands-on
- Scaffold prompt
- v1 → v2 improvement loop

Learning Outcomes:

By the end of this module, you will be able to:

- 01 Understand the AI coding landscape to select the right tool for a given task
- 02 Distinguish Claude Code from chat tools and navigate its interface using key commands
- 03 Write a scaffold prompt that generates useful, first-attempt code from a plain-English brief
- 04 Apply an iterative prompting loop to improve output quality across multiple attempts

Before We Start

Claude Code is pre-installed — no setup today

This is a hands-on intensive. You will write and run real code all day, starting in Exercise 1

When something breaks, that is where learning happens — don't skip the error

Keep a running prompt log as you work — you will use it in Part 3

Questions are welcome throughout — raise your hand or drop them in chat

The learner who asks the most questions today gets a special shout-out in the next module

Meet Your Instructor

01 THE AI CODING LANDSCAPE

What's Changed — And What Hasn't

What's Changed

- The speed at which working code can be produced
- How much one engineer can ship across a multi-file codebase
- The cost of a first working draft
- How quickly you can navigate an unfamiliar codebase

What Hasn't Changed

- Syntax still matters — garbage in, garbage out
- Architecture decisions still require human judgement
- Invariants, edge cases and failure modes still need you to specify them
- Knowing when to stop — the AI is a skill, not a setting

The AI Coding Landscape: Four Platform Tiers

T1

App Builders



FOR

Non-tech founders, designers, PMs

WHAT IT IS

Browser based. Prompt-to-app. UI plus backend plus hosting in one flow.

REPRESENTATIVE TOOLS

Lovable, Bolt.new, v0, Base44, Replit Agent

T2

AI IDEs



FOR

Working developers, technical founders

WHAT IT IS

AI-augmented code editors. You own the code. Multi-file edits, codebase awareness.

REPRESENTATIVE TOOLS

Cursor, Windsurf, GitHub Copilot, Trae

T3

Terminal Agents



FOR

Senior engineers, platform teams

WHAT IT IS

CLI-native agents. Operate on real repos. Highest raw capability on complex codebases.

REPRESENTATIVE TOOLS

Claude Code, Gemini CLI, OpenAI Codex, Devin

T4

Spec-Driven Agents



FOR

Enterprise teams, regulated domains

WHAT IT IS

Spec-first orchestration. Validation gates. Agent execution against versioned specs.

REPRESENTATIVE TOOLS

AWS Kiro, GitHub Spec Kit, OpenSpec, Augment Intent

The Right Mental Model

AI coding tools are like a very fast junior developer. The speed is theirs. The judgement is yours.



They write quickly.



They hallucinate confidently.



They need direction.

The output is only as good as the brief — and only as safe as your review.

Tool-by-Tool: Best For and Limitation

TOOL	TIER	BEST FOR	LIMITATION
Lovable	T1	Full-stack scaffolding. Polished UI defaults.	<i>Vision over surgical refinement. Web only.</i>
Bolt.new	T1	Fastest browser prototype. Live preview. Mobile via Expo.	<i>SPA bias. Vague prompts produce incomplete builds.</i>
v0 (Vercel)	T1	React + shadcn/ui. Fastest Vercel deploy.	<i>Frontend leaning. Less for backend or non-coders.</i>
Replit Agent	T1	Cloud IDE plus agent. Multi-language. Learning friendly.	<i>Infinite-fix loops burn credits. Non-coders see raw code.</i>
Cursor	T2	Multi-file edits. Deep codebase awareness. Rules files.	<i>No persistent spec memory. Rules are advisory, not enforced.</i>
Windsurf	T2	Cascade agent for large codebases. .windsurfrules.	<i>Start fresh chat per feature. Smaller mindshare than Cursor.</i>
Claude Code	T3	Agentic coding. Plan mode. Highest benchmark scores.	<i>Terminal native. Requires strong prompting discipline.</i>
GitHub Copilot	T2	Inline autocomplete in VS Code, JetBrains.	<i>Less autonomous. Different category from agentic tools.</i>
Kiro (AWS)	T4	Spec-first IDE. Validation gates. Audit trail.	<i>Newer. Best for enterprise and regulated workloads.</i>
GitHub Spec Kit	T4	Specify-Plan-Tasks-Implement team workflow.	<i>Requires team buy-in to spec discipline.</i>
OpenSpec	T4	Solo SDD via /openspec commands in Cursor.	<i>Best for 2-3 session features, not large team flows.</i>
Augment Intent	T4	Living specs. Coordinator + verifier agents.	<i>Heaviest setup. Pays back on team-scale projects.</i>

Which Tool Would You Reach For?

REFLECT

Three scenarios. Pick the right tool for each. Vote — then we discuss why.

SCENARIO A

You need a clickable UI mock of the clinician review console for a design review on Friday. Nobody will run the backend.

SCENARIO B

You have inherited a 3,000-line Python service. A bug is breaking the payment flow. You need to find the source and fix it without touching anything you don't understand.

SCENARIO C

You are writing a new data validation function and want line-by-line suggestions as you type, inside your existing editor.

Claude Code

GitHub Copilot

Cursor / Devin

Lovable / Bolt / Emergent

A Quick Recap

What you now have

- ✓ A map of the four AI coding tool categories and where each one belongs
- ✓ The mental model: fast junior developer — you supervise, you review, you decide
- ✓ A framework for choosing the right tool before you start

Coming Up — Part 2: Getting Started with Claude Code

02 GETTING STARTED WITH CLAUDE CODE

Claude Code vs. Chat — The Key Difference

Chat Interface (Claude.ai, ChatGPT)

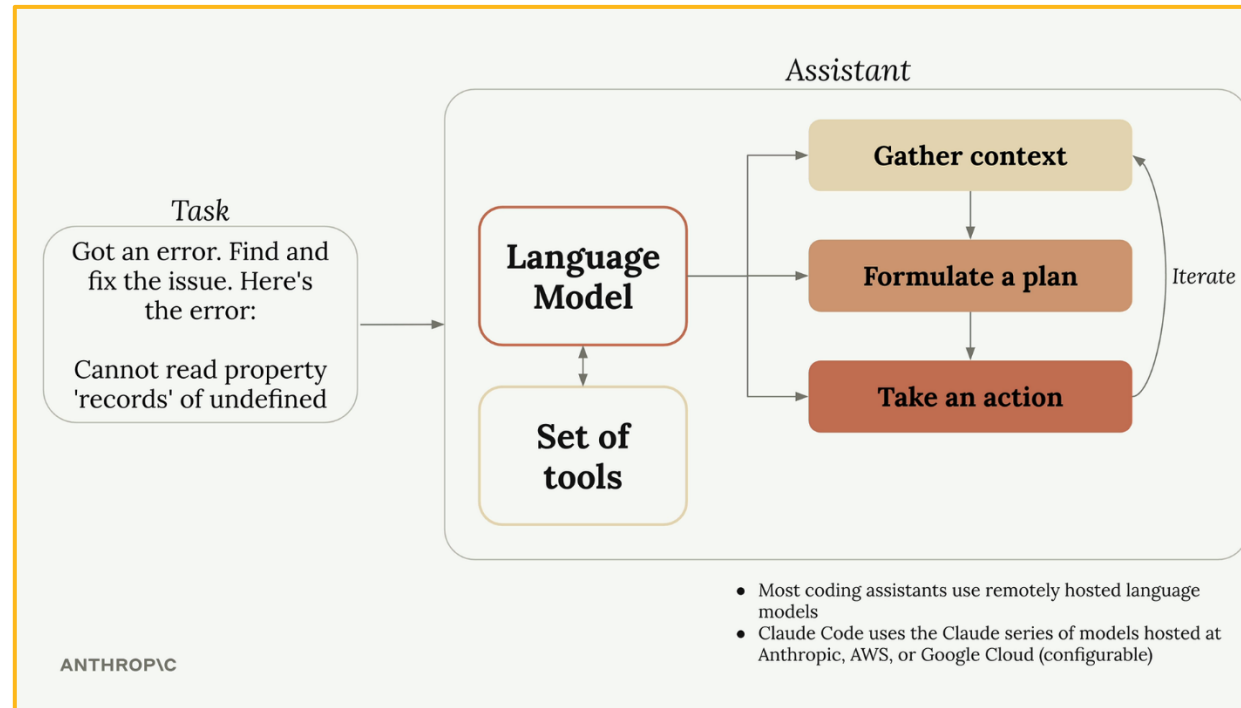
- You paste code in. Claude responds in text. You copy the output back manually.
- Claude has no access to your files.
- Every conversation starts fresh — no memory of your project.
- Best for: **questions, explanations, one-off tasks.**

Claude Code (Terminal)

- Claude runs in your terminal and reads your files directly.
- It proposes changes as diffs — you accept or reject each one.
- It can execute commands on your machine, with your approval.
- Best for: **building, editing, and navigating real projects.**

THE CRITICAL DIFFERENCE: Claude Code acts on your project (new or existing). A chat tool advises on it. One reads your files. The other doesn't know they exist.

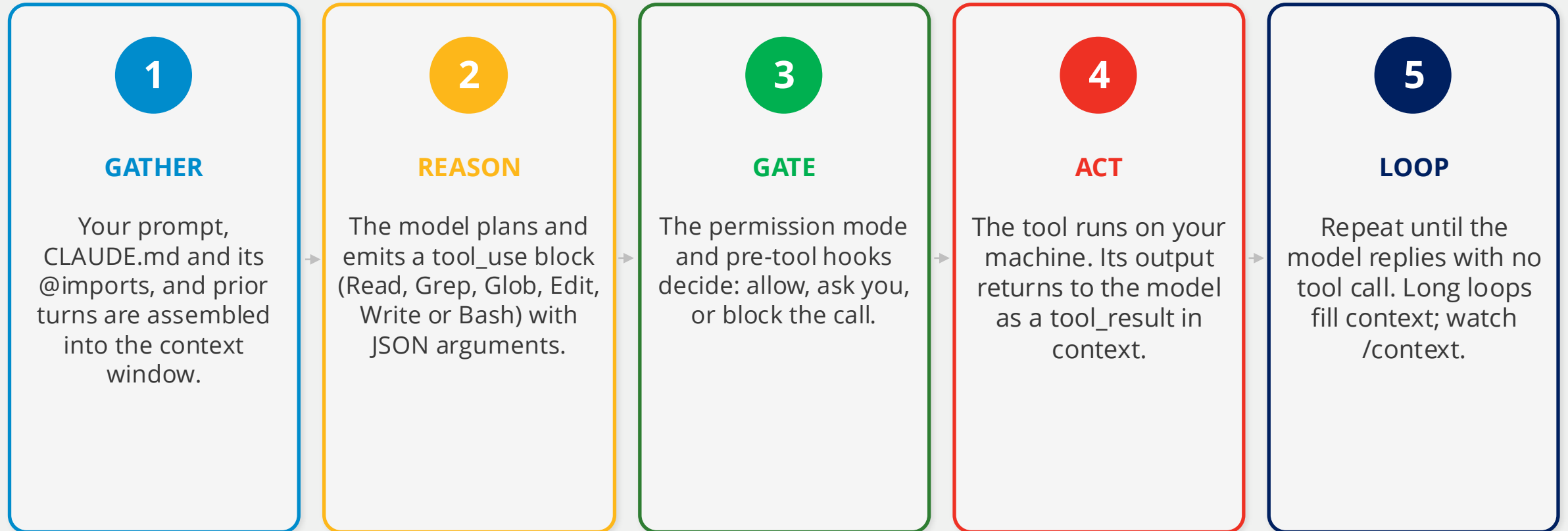
How Coding Assistants Work



A coding assistant is more than just a tool that writes code - it's a sophisticated system that uses language models to tackle complex programming tasks. Understanding how these assistants work behind the scenes will help you appreciate what makes a truly powerful coding companion.

Under the Hood: The Agent Loop

Claude Code is a model in a loop with tools. Every action you approve is one iteration.



Everything Claude "knows" about your repo arrived through a tool call you can see. If it did not read it, it is guessing.

That Doesn't Mean It Has All The Permissions

What Claude Does - Autonomous

READ

Claude reads your files and scans your project structure without asking. This is how it understands context.

You see: file names listed as Claude scans them.

PROPOSE

Claude generates code changes and shows them as diffs. It writes — but does not save — until you decide.

You see: the proposed change before anything is applied.

Where Claude Requires your approval

EDIT

Claude applies changes to your files only after you confirm. Press Y to accept, N to reject.

You control: every write to disk.

EXECUTE

Claude runs terminal commands only after showing you what it intends to run and waiting for your go-ahead.

You control: every command that runs on your machine.

THE PRINCIPLE: Claude proposes. You decide. Nothing changes on your machine without your explicit approval.

Context Is Everything for Coding Assistants

You can run Claude Code. You know the commands. But Claude still knows nothing about your project.

- Claude Code is not a magic box — it works on exactly what you give it
- Every session starts cold — no memory of what you built yesterday, no knowledge of what not to touch, no understanding of how your team works
- Give it a vague brief and it produces reasonable-looking code that solves the wrong problem
- Give it a precise brief and it becomes the fastest developer you have ever worked with

The quality of what you get out is directly proportional to the quality of the context you put in — and that is exactly what CLAUDE.md is built to solve.

Context Engineering

The context window is a token budget. Spend it on the tokens that change the answer.

MEMORY HIERARCHY

- Project `./CLAUDE.md` is shared through git;
 `~/claude/CLAUDE.md` holds personal defaults
- `@path` imports pull detail in on demand, e.g.
 `@docs/SPEC.md`

WHAT FILLS IT

- System prompt and tool schemas, including every connected MCP tool
- Memory files, file reads, command output and the conversation itself

MANAGING IT

- `/context` shows usage by category; `/cost` shows spend
- `/compact <focus>` summarises and continues;
 `/clear` starts from zero

FAILURE MODES

- Stale or contradictory `CLAUDE.md` rules win silently
- Huge tool output (logs, JSON dumps) crowds out the spec: `grep`, `do not cat`

Rule of thumb: a short `CLAUDE.md`, a detailed `SPEC.md` via `@import`, and `/compact` at natural checkpoints.

Before Your Real Project — A Practice Round

You are about to write your first CLAUDE.md. To do that, you need a project. This one is a throwaway — it exists only to practice the skill.

The Scenario — Log Triage CLI

- Step 1 - Generate app.log.jsonl (40-60 lines: ts in ISO-8601, level, endpoint, exception_type, latency_ms; a mix of INFO, WARN and ERROR over the last 48 hours)
- Step 2 - A Python CLI that reads the log, groups ERROR lines by exception_type and endpoint, and prints the top 5 with count, first seen and last seen. Flag --since 24h.

Three elements below the scenario card (small, three columns):

What it does	What it does NOT do	Why we're using it
Parses JSONL, filters by time, aggregates, prints	No alerting, no writes, no UI	Simple enough to brainstorm in 3 minutes. Generic enough that everyone builds the same thing.

What is **CLAUDE.md**?

CLAUDE.md is a plain text file at your project root. Claude reads it automatically at the start of every session — before you type a single prompt. It is your instruction manual to Claude: what the project is, how it works, and what not to touch.



Onboarding doc

Like the README you wish someone had written before you joined.



Project memory

Claude forgets between sessions — CLAUDE.md restores context instantly.



Team contract

Coding conventions, naming rules, no-go zones — enforced by context. You keep updating it as the project evolves.

CLAUDE.md — The 5-Section Anatomy

1. Project Overview

What does this project do?
Who uses it?

"Patient appointment scheduling system for NHS clinic staff."

2. Tech Stack

Languages, frameworks, DB, libraries. Versions matter.

"Flask 3.0 · SQLAlchemy · SQLite · Python 3.11 · Entry: app.py"

3. Coding Conventions

Naming rules, response format, ORM-only SQL.

"snake_case everywhere. Routes return JSON: {data, error, status}."

4. Do-Not-Touch Zones

Files or logic that must not be modified.

"Do not edit db/schema.sql directly. Do not remove /health route."

5. Useful Context

Anything a new engineer would need to know.

"conflict.py boundary is intentionally strict — do not loosen."

Common **CLAUDE.md** Mistakes

X Using # for section headers



✓ Use ## (H2) — Claude parses sections by H2 reliably.

X Vague project description



✓ Include what the project does, who uses it, and the entry point.

X No do-not-touch section



✓ Always specify what Claude must never change — saves expensive mistakes.

X Writing CLAUDE.md once and forgetting it



✓ Update after each module as you discover new things.

X Pasting CLAUDE.md content into the prompt



✓ Claude reads CLAUDE.md automatically — don't duplicate.

Key Commands

Remember these commands, we will use them shortly.

Commands	What they do	Use when
/init	Scans your project and generates a starter CLAUDE.md automatically.	Starting on a new project for the first time.
/compact	Summarises the conversation so far and continues with a shorter context. Prevents Claude from losing track on long sessions.	You have been working for a while and responses are becoming less accurate.
/cost	Shows how many tokens you have used and the cost of the current session.	You want to monitor spend or sense you are approaching a context limit.
/clear	Starts a completely fresh conversation. Claude forgets everything in the current session.	Responses have drifted and you want a clean slate.

Write your first **CLAUDE.md**

HANDS-ON LAB

- 1 Read the scenario** — account tracker, CSV input, terminal output, no writes, no UI
- 2 Brainstorm with plain prompts** — open Claude Code and ask: "What would this script need to do, and what should it never do?" Read what comes back.
- 3 Generates a starter CLAUDE.md** from the project context

Shop API

What this is

An e-commerce backend. It does shopping stuff.

BAD: "shopping stuff" describes nothing. No user journey, no scope boundary.

Claude can't tell what's in scope, so it'll invent features.

Stack

- Python (recent version)

- A database

- Whatever web framework is cleanest

BAD: no pinned versions, no named framework, no ORM, no test runner.

"Whatever is cleanest" hands an architecture decision to the model

and guarantees inconsistency between sessions.

Conventions

- Write good, clean, modern, production-grade code.

- Follow best practices.

- Handle money correctly.

- Make it fast and scalable.

BAD: every line is a vibe, not a rule. "Best practices" is unverifiable.

"Handle money correctly" — as float? as Decimal? as minor units? Pick one.

"Fast and scalable" on a PoC invites premature caching/queues nobody asked for.

Notes

- Be creative and use your judgment.

- If something is missing, just add it.

- Move fast, we'll clean it up later.

BAD: this is the opposite of a guardrail. It explicitly licenses scope creep,

silent dependency additions, and unreviewed multi-file rewrites.

Run

- Just run it the normal way.

BAD: there is no "normal way." No run command, no test command.

The one section that should be copy-pasteable is useless.

OrderDesk API

What this is

A small storefront ordering and inventory service. PoC, not production.

One user journey: browse the catalog, place an order, reserve stock, mark it paid or cancelled.

Stack

- Python 3.12, FastAPI, Uvicorn
- SQLAlchemy ORM on SQLite (file: orderdesk.db) for the PoC
- httpx for the external payment-intent call (mock gateway)
- pydantic v2 for request and response models
- pytest for tests

Conventions

- Layout: app/main.py, app/db.py, app/models.py, app/schemas.py, app/routes.py
- Type hints on every function. Docstrings on every endpoint.
- Money is stored as integer minor units (paise / cents), never float.
- Prices live on the catalog item. Order lines snapshot the price at order time and never recompute from the live catalog.
- Stock moves through a reservation, never edited in place. Reserve on order create, release on cancel, commit on pay.
- Order create is idempotent on an Idempotency-Key header. The same key returns the same order, never a duplicate.
- Never oversell. Reject the order if reservable stock is below the requested quantity.
- Never hardcode secrets. Read them from environment variables via python-dotenv.

Run and test

- Run: `python -m uvicorn app.main:app --reload`
- Test: `python -m pytest -q`

Do not touch

- Do not edit .env, .git, or orderdesk.db directly.
- Do not add new third-party dependencies without telling me first.
- Do not invent endpoints, statuses, or order states that are not in the brief.
- Do not bypass the reservation flow to adjust stock counts directly.

Always show me a diff and wait for approval before editing more than one file at a time.

Let's Test Our Understanding

QUICK CHECK

Q1

Which CLAUDE.md section lists files or logic that should never be modified?

Tech Stack

Do not touch zones

Coding Conventions

Useful context

Q2

Which two actions can Claude perform autonomously, without asking for approval

Edit and Execute

Read and Propose

Propose and Execute

Read and Edit

Let's Test Our Understanding

QUICK CHECK

Q1

Which CLAUDE.md section lists files or logic that should never be modified?

Tech Stack

Do not touch zones

Coding Conventions

Useful context

Q2

Which two actions can Claude perform autonomously, without asking for approval

Edit and Execute

Read and Propose

Propose and Execute

Read and Edit

A Quick Recap

What you now have

- ✓ A clear understanding of how Claude Code differs from a chat tool
- ✓ Hands-on experience navigating the interface and issuing real prompts
- ✓ A CLAUDE.md started for the workshop project
- ✓ The key commands and when each one matters
- ✓ The permission model — nothing changes on your machine without your approval

Coming Up — Part 3: Prompt Engineering for Code Generation

03 **PROMPT ENGINEERING FOR CODE GENERATION**

How Coding Prompts Differ from General Prompts

General Prompts

- Work well with vague instructions — Claude fills gaps from general knowledge
- Output is text — easy to read and evaluate
- Ambiguity produces a reasonable response
- *"Explain what a vector database is"*

Coding Prompts

- Require named specifics — Claude fills gaps from training data, which may be outdated or wrong
- Output is executable — every gap becomes a potential bug
- Ambiguity produces plausible code that solves the wrong problem
- *"Fix the rescheduling bug" → Claude does not know which file, which function, or what "fixed" means*

The Scaffold Prompt Template

[Stack] + [Architecture Intent] + [Edge Cases] + [Output Format]

Stack

What language, framework, and libraries should Claude use?

"Python 3.11 · Flask · SQLAlchemy · Entry point: app.py"

Architecture Intent

What should this do and how should it fit into the project?

"Add a validation function that checks whether a submitted date range overlaps with any existing entries in the database."

Edge Cases

What should happen when things go wrong?

"If the date range is invalid, return 422 with an error message. If the database is unreachable, log the error and return 500."

Output Format

How should the result be structured?

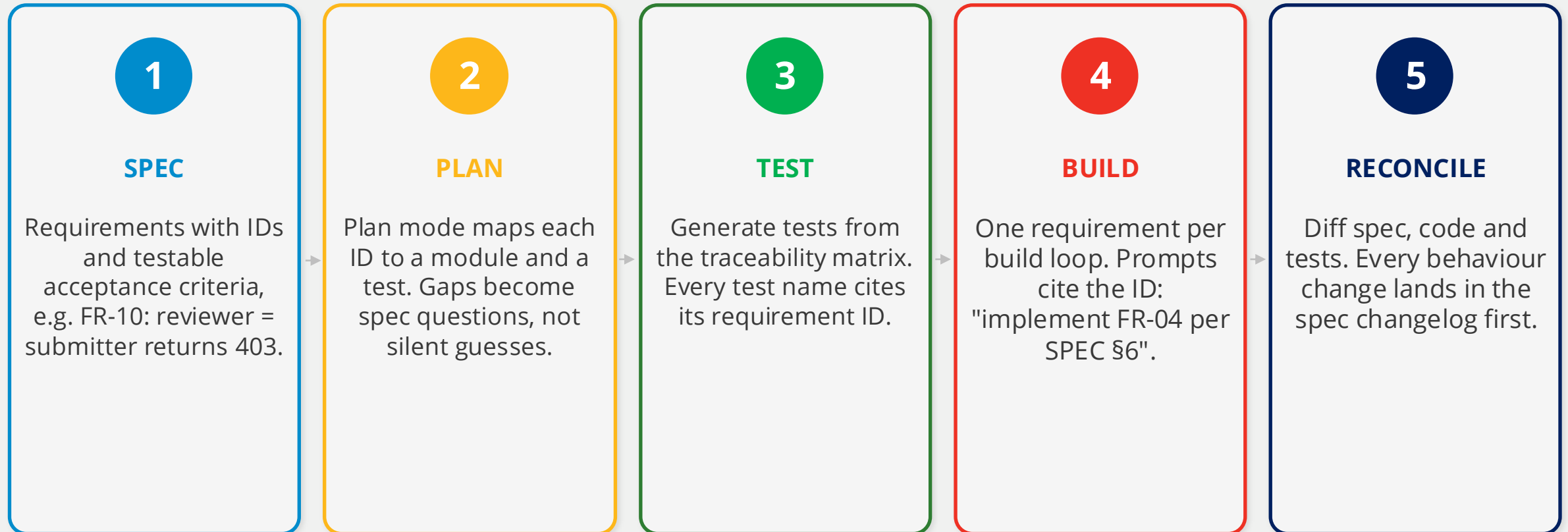
"Return JSON: {data, error, status}. Add a Google-style docstring to the function."

Assembled:

"In Python 3.11 using Flask and SQLAlchemy, add a validation function to utils/validation.py that checks whether a submitted date range overlaps with any existing entries. Return 422 on invalid range, 500 on database failure. Return JSON: {data, error, status}. Include a Google-style docstring."

Spec-Driven Development

The spec, not the chat history, is the memory of what was agreed.



Prompts are disposable. Specs, tests and CLAUDE.md are the durable artefacts that steer every future session.

Weak Prompt vs. Strong Prompt

✗ Weak Prompt

"Write a script that cleans our training data."

✓ Strong Prompt

"In Python 3.12, write a CLI that reads data/train.jsonl, drops records with an empty text field or a duplicate id, normalises whitespace, and writes data/train.clean.jsonl. Log dropped-row counts by reason. Add a --dry-run flag."

What Claude does not know:

- Where the data comes from — API? CSV? database?
- What language or libraries to use
- What a "report" means — HTML? PDF? printed to terminal?
- What "clean" means — dedupe? PII removal? schema checks?
- What to do if data is missing or malformed

What Claude now knows:

- Language and version ✓
- Data source and location ✓
- What the output looks like ✓
- How to handle errors ✓
- What the user controls ✓

The Iterative Prompting Loop

Treat each Claude Code response as one step in a build loop — not a finished answer.

STEP 1 — Rough Scaffold

Prompt for structure first. Get the skeleton. Do not expect working code yet.

"Scaffold a Flask route handler for submitting a new entry. Structure only — no business logic."

STEP 2 — Working Function

Add the logic. Run it. See what breaks.

"Now add the validation logic. The entry must not overlap with any existing date range."

STEP 3 — Tested Module

Ask Claude to write tests. Run them. Fix what fails.

"Write pytest tests covering: valid input, overlapping date range, and missing required fields."

STEP 4 — Documented Code

Ask for docstrings and inline comments. Review them before you commit.

"Add a Google-style docstring and inline comments explaining the non-obvious logic."

The Habit: Never expect step 4 quality from a step 1 prompt. Each prompt is one move. The loop is the method.

Prompting for Understanding, Not Just Output

The most important habit for anyone who needs to review, sign off on, or stand behind AI-generated code.

The Prompt:

"Explain what this does, line by line."

Why It Matters:

You cannot review what you do not understand. If you accept a diff you cannot explain, you own the consequences when it breaks.

Prompting for Understanding, Not Just Output

Three Levels of Understanding Prompts

1

LINE BY LINE For unfamiliar code

*"Explain this function line by line.
Assume I understand the basics but
not this specific library."*

2

DECISION RATIONALE For architecture choices

*"Why did you structure it this way?
What would break if I changed the
approach?"*

3

RISK IDENTIFICATION Before you commit

*"What are the three most likely ways
this could break in production?"*

THE RULE: *If you cannot explain what Claude wrote, you are not ready to commit it. Understanding prompts are not a slowdown — they are how you stay accountable for what you ship.*

Five Prompting Techniques to Master

Master these and the choice of platform becomes secondary.



One-shot

WHEN: Simple, bounded task

Single focused prompt with full context. Adding a function, a component, a fixed transformation.



Multi-shot

WHEN: Style or architecture matters

Provide multiple worked examples that demonstrate the desired pattern. The AI imitates them.



Task Decomposition

WHEN: Large multi-file features

Break the feature into 3-5 smaller focused prompts. Each delivers a coherent slice with done criteria.



Chain-of-Thought

WHEN: Logic heavy or algorithmic work

Ask the AI to reason stepwise through the logic before writing code. Catches design errors before code.



Metaprompting / Skills

WHEN: Repeated team patterns

Save reusable prompt templates as .md files (SKILL.md, .cursorrules). Apply on demand.

Plan-first prompting: never let AI write code without first seeing and approving a plan.

Common Prompting Mistakes

1 — Under-specifying context

"Fix the bug" without naming the file, function, or what correct behaviour looks like.
→ Claude fixes the wrong thing — confidently.

2 — Ignoring error handling

Not telling Claude what to do when inputs are missing, malformed, or a service is unavailable.
→ Code works in demos. Breaks in production.

3 — Accepting the first output without running it

Reading the code, deciding it looks right, and moving on.
→ Code that looks correct and code that runs correctly are not the same thing. Always run it.

4 — Letting Claude choose the architecture

Not specifying where new code should live, how it should integrate, or what patterns to follow.
→ Claude makes reasonable decisions that conflict with decisions you already made elsewhere.

5 — Treating each prompt as a one-shot

Expecting a single prompt to produce complete, tested, documented, production-ready output.

Write a Brief, Generate v1, Improve to v2

Same project from Part 2. Your CLAUDE.md is already written — now write the prompts that put it to work.

1

Part 1: Generate Sample Data

Create `app.log.jsonl` with 40–60 JSON lines so the CLI has realistic data. Fields: `ts` (ISO-8601 UTC), `level`, `endpoint`, `exception_type`, `latency_ms`.

2

Part 2: The Task

Write a Python CLI (`log_triage.py`) that:

Streams `app.log.jsonl` line by line; skips and counts malformed lines.

Filters to ERROR lines newer than `--since` (default 24h), using timezone-aware datetimes.

Groups by (`exception_type`, `endpoint`) and computes `count`, `first_seen`, `last_seen`, `p95 latency_ms`.

Prints a formatted terminal table of the top 5 groups, showing:

Exception / Endpoint

Count and p95 latency

First and Last Seen

Sorts by count descending; exits non-zero if any group exceeds `--threshold`.

Write pytest cases (malformed lines, empty window, ties), run the suite, report results

3

Part 3: Compare earlier to current solution and list pros / cons

THE QUESTION:

DISCUSSION

What was the single change between your v1 prompt and your v2 prompt that produced the biggest quality difference in the output?

Which scaffold component made the biggest difference — stack, architecture intent, edge cases, or output format?

Recap - What You Can Do Now

THE AI CODING LANDSCAPE

- Map the four tool categories and where each belongs
- Apply the selection framework — right tool for the right task
- Supervise AI output with the correct mental model

CLAUDE CODE

- Navigate the terminal interface and issue real prompts
- Use CLAUDE.md to give Claude reliable project context
- Use plan mode, /compact, and /cost with confidence
- Understand the permission model — nothing changes without your approval

PROMPT ENGINEERING

- Write a scaffold prompt using stack, architecture intent, edge cases, and output format
- Apply the iterative loop — rough scaffold through to documented code
- Prompt for understanding, not just output
- Diagnose the gap between v1 and v2 and close it deliberately

Practice

Task 1

Pick one small utility you need. Write a scaffold prompt using the template. Generate v1. Run it.

Task 2

Review v1 with fresh eyes. Identify the gaps. Rewrite the prompt. Generate v2. Note what changed.

Task 3 Write a one-paragraph brief for DischargeFlow in your own words, then compare it with docs/BRIEF.md in the course repo.

Task 4

Open Claude Code on a project you already own. Run /init. Read the CLAUDE.md it generates. Edit one section to make it more accurate.

What's Next: Building Your First PoC

- Scoping and Briefing Your PoC
- The Build Loop: From Brief to Running Code
- Finishing, Documenting and Stabilising



Thank You.

CellStrat

Claude Code Certified Architect (CCA) · a hands-on
CellStrat intensive. Build, break, secure, test and ship.



CellStrat